DSA I (CSE 2215)

Selection Sort and Counting Sort

Asif Abrar, Lecturer of CSE

UIU

Overview

- Selection Sort (Code + Simulation + Complexity)
- Counting Sort (Code + Simulation + Complexity)

Selection Sort – Code

```
void selectionSort(int arr[], int n) {
    for(int i = 0; i < n-1; i++) {
        int minIndex = i;
        for(int j = i+1; j < n; j++) {
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
}
```

Selection Sort – Simulation

Initial Array: 7 4 1 9 3

Pass	Action	Array State
1	Find min in [7,4,1,9,3] $\rightarrow$ 1	1 4 7 9 3
2	Find min in [4,7,9,3] $\rightarrow$ 3	1 3 7 9 4
3	Find min in [7,9,4] $\rightarrow$ 4	1 3 4 9 7
4	Find min in [9,7] $\rightarrow$ 7	1 3 4 7 9

Selection Sort – Time Complexity

- Always scans remaining array to find minimum.
- **Best Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Space Complexity:** $O(1)$
- Not stable (swaps may break order)

Counting Sort – Code (Part 1)

```
void countingSort(int arr[], int n) {

    // Step 1: Find maximum value
    int maxVal = arr[0];
    for(int i = 1; i < n; i++) {
        if(arr[i] > maxVal)
            maxVal = arr[i];
    }

    // Step 2: Create count array
    int count[maxVal + 1] = {0};

    // Step 3: Frequency Count
    for(int i = 0; i < n; i++) {
        count[arr[i]]++;
    }
```

Counting Sort – Code (Part 2)

```
    // Step 4: Prefix Sum (Cumulative count)
    for(int i = 1; i <= maxVal; i++) {
        count[i] += count[i - 1];
    }
    // Step 5: Build the output array (stable)
    int output[n];
    for(int i = n-1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
    // Step 6: Copy back
    for(int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}
```

Counting Sort – Step-by-Step Simulation

Input Array: 4 1 3 4 2

Step 1: Find Maximum

max = 4

Step 2: Initialize Count Array

Size = max + 1 = 5

Index: 0 1 2 3 4
Count: 0 0 0 0 0

Counting Sort – Frequency Count

Input: 4 1 3 4 2

Step 3: Count Occurrences

- 4 $\rightarrow$ count[4]++
- 1 $\rightarrow$ count[1]++
- 3 $\rightarrow$ count[3]++
- 4 $\rightarrow$ count[4]++
- 2 $\rightarrow$ count[2]++

Count Array Now:
Index: 0 1 2 3 4
Count: 0 1 1 1 2

Counting Sort – Prefix Sum Breakdown

Current Count: 0 1 1 1 2

Step 4: Prefix Sum

count[i] = count[i] + count[i-1]

- count[1] = 1 + 0 = 1
- count[2] = 1 + 1 = 2
- count[3] = 1 + 2 = 3
- count[4] = 2 + 3 = 5

Counting Sort – Prefix Sum Meaning

Prefix Sum Array

Index:	0	1	2	3	4
Count:	0	1	2	3	5

Meaning of Prefix Sum

- `count[i]` = total numbers $\leq i$
- So numbers $\leq i$ fill output positions 0 to `count[i]` $- 1$

Breakdown (What each value means)

- **count[1] = 1** $\rightarrow$ all 1's end at index 0
- **count[2] = 2** $\rightarrow$ all 2's end at index 1
- **count[3] = 3** $\rightarrow$ all 3's end at index 2
- **count[4] = 5** $\rightarrow$ all 4's end at index 4

Counting Sort – Building Output Array

Step 5: Process input in reverse

Input: 4 1 3 4 2

Prefix Count: 0 1 2 3 5

Element	Action
2	output[1] = 2
3	output[2] = 3
1	output[0] = 1
4	output[4] = 4
4	output[3] = 4

Counting Sort – Final Output

Output Array:

1 2 3 4 4

Counting sort is:

- Stable
- Not in-place

Counting Sort – Time Complexity Breakdown

Let:

- n = number of elements
- k = max element

Time Complexity

- Find max: (n)
- Init count array: (k)
- Frequency count: (n)
- Prefix sum: (k)
- Build output: (n)

$$T(n, k) = \Theta(n + k)$$

Counting Sort – Best / Worst / Average Case

Best Case: $(n + k)$ — all steps required
Worst Case: $O(n + k)$
Average Case: $(n + k)$

Thank You!

Questions?